

✓ Solving the OneMax problem using a GA

✓ Constants and Imports

We will need various constants such as the population size, crossover reate, mutation rate etc. along with imports for the packages we are going to make use of. Typically these are passed as parameters to the algorithms but we're keeptrying to keep things simple for the time being.

```
import string
import random
random.seed(42) # initialise and make repeatable
from operator import itemgetter # may be useful in retrieving elements of the population

#Population size and other constants - these can be changed.
pop_size=50
crossover_rate = 0.75
mutation_rate = 0.05
```

Representation

Always the first thing to think about - how do we represent a solution to the problem - i.e. how should this be encoded and what data type (or types) should be used. It can often help to think about how you might manipulate this (to generate new solutions) and evaluate it. However this case is easy as the problem and the representation are identical - a string of 1's and 0's - so nothing else needs to be done here.

✓ Fitness Function

This defines how we score or evaluate a solution. You can make use of the target string at this point.

The fitness function has an important role is guiding the search towards the solution, so it is essential that we are able to identify when one solution is better than another.

There are various alternatives but the more information you provide the quicker the search is likely to be.

You will already have defined a fitness function in the previous exercise and can just reuse it here.

```
# Function to calculate a candidates solution fitness - solutions closer to the goal should score higher
# Input parameters: candidate solution (string of 1's and 0's)
# Returns: fitness of the candidate solution
def fitness(solution):
    # Your code goes here
    return solution.count('1')
```

```
# test the above (precise results will depend on your function and may vary with string length)
print(fitness("1111111111")) # output should be the highest score
print(fitness("1010110010")) # output should be a mid-range score
print(fitness("0000000000")) # output should be the lowest score
print(fitness("11111")) # output should be the highest score
print(fitness("10101")) # output should be a mid-range score
print(fitness("00000")) # output should be the lowest score
print(fitness("1")) # output should be the highest score
print(fitness("0")) # output should be the lowest score
```

```
10
5
0
5
3
0
1
0
```

The GA algorithm

The main steps are:

- Generate a random population
- Evaluate the fitness of the population
- While a solution has not been found
 - Select and crossover individuals to create a new population

- Mutate some individuals of the new population
- Make the new population the current population
- Evaluate the fitness of the current population
- Return the best individual and its fitness

✓ Generate a random population

Store the population as a list of tuples so that we can associate a fitness value with each element.

We also need a function to generate a random binary string .

✓ Generate an Individual

Create a random string of 1's and 0's of length defined in the parameter. Again you should already have a function which does this.

```
# Generate a random string of 1's and 0's as defined by the parameter
# Input parameters: length - the length of the string to generate
# Returns: The randomly generated string
def gen_individual(length):
    return ''.join(random.choice('01') for _ in range(length));

# test... run this a few times to check the output is as expected
print(gen_individual(1))
print(gen_individual(5))
print(gen_individual(10))
```

```
0
01000
0010000000
```

✓ Create the population

Using the `gen_individual()` function above, build a population from individuals which are represented as lists [string, fitness]. The fitness will be inially 0 as we have not calculated it yet. The population itself will be a list of these lists.

i.e. `population = [ ["individual1", fitness1], ["individual2", fitness2], ["individual3", fitness3], ... etc.]`

```
# Input parameters: none
# Returns: A population of randomly generated individuals with associated fitness values (set to 0)
def gen_population():
    population = []
    for _ in range(pop_size):
        individual = gen_individual(10)
        population.append([individual, 0])
    return population

# Your code goes here

# test
print(gen_population())
```

```
[['1011001110', 0], ['0100101110', 0], ['1010110000', 0], ['1000111101', 0], ['1010000111', 0], ['0100011100', 0], ['
```

✓ Evaluate the population

Go through each element in the population, calculating its fitness and associating the fitness value with the individual in the tuple.

We will also need a function that returns the best individual in the population and its associated fitness.

```
# Input parameters: A population of individuals (solutions)
# Returns: The population modified with the computer fitness of each individual
def evaluate_pop(pop):
    for individual in pop:
        score = fitness(individual[0])
        individual[1] = score
    return pop

# Your code goes here

# Test
newpop = gen_population()
print(evaluate_pop(newpop))
```

```
[[ '100000000', 1], ['0100111000', 4], ['1001110000', 4], ['1001110101', 6], ['1011101011', 7], ['1111001101', 7], ['
```

```
# Input parameters: A population of individuals (solutions)
# Returns: The fittest individual (with its score)
def fittest_individual(pop):
    # Your code goes here
    return max(pop, key=itemgetter(1))

#test
newpop = gen_population()
print(fittest_individual(newpop))
```

```
['0010000010', 0]
```

Selection

The selection of candidate parent solutions is going to be very simple - we are going to randomly select from the top 50% of solutions. Selection is with replacement so there is a chance that some individuals may be chosen more than once whereas others may never be selected as parents. Remember the population size needs to remain constant over generations so the number of parents should be equal to the size of the population.

```
# orders the list according to fitness and returns the top 50%

# Input parameters: A population of individuals
# Returns: The selected parents drawn from the top 50% of the individuals (as ranked by fitness)
def find_top_50(pop):
    sorted_pop = sorted(pop, key=itemgetter(1), reverse=True)
    number_of_parents = int(len(sorted_pop) / 2)
    return sorted_pop[:number_of_parents]

#test
newpop = gen_population()
print(find_top_50(newpop))
```

```
[[ '1110000110', 0], ['1110110000', 0], ['1010110110', 0], ['0100000101', 0], ['1100011111', 0], ['0010111000', 0], ['
```

Recombine individuals to create the next generation

This step involves two sub-steps - the identification of pairs of parents and recombination to create offspring.

- The `select_and_generate_new_population` function repeatedly (and randomly) selects pairs of parents from the previously selected pool of parents which are recombined to form new individuals (children) which will be used to create the new population.
- Part of this will also involve defining a function to perform the crossover operation.

```
# Input parameters: Two parents (tuples of the form <individual, fitness>)
# Returns: Two children after combining the two parents (tuples of the form <individual, 0>)
# or the original two parents if the crossover threshold is not met
def crossover(parent1, parent2):
    if random.random() < crossover_rate:
        p1_string = parent1[0]
        p2_string = parent2[0]
        length = len(p1_string)

        crossover_point = random.randint(1, length - 1)

        child1_string = p1_string[:crossover_point] + p2_string[crossover_point:]
        child2_string = p2_string[:crossover_point] + p1_string[crossover_point:]

        return [child1_string, 0], [child2_string, 0]
    else:
        return [parent1[0], 0], [parent2[0], 0]

    # Your code goes here

# test
print(crossover(("11001100",4),("10101010",4)))

# Input parameters: a population
# Returns: The new population after selection and crossover have been applied
def select_and_generate_new_population(parents):
    # Your code goes here
    new_population = []
    # Loop until the new population reaches the desired size
    while len(new_population) < pop_size:
```

```

# Randomly select two parents from the parent pool
parent1, parent2 = random.sample(parents, 2)

# Perform crossover to get two children
child1, child2 = crossover(parent1, parent2)

# Add the new children to our new population
new_population.append(child1)
new_population.append(child2)

return new_population

# test
newpop = gen_population()
select_and_generate_new_population(newpop)

```

```

(['11001010', 0], ['10101100', 0])
[['0111001111', 0],
 ['0001111100', 0],
 ['1000100111', 0],
 ['0001001111', 0],
 ['1100001100', 0],
 ['1100000110', 0],
 ['0010110110', 0],
 ['1100001011', 0],
 ['0110101011', 0],
 ['0001111110', 0],
 ['1011111110', 0],
 ['0011010100', 0],
 ['1011110111', 0],
 ['1110100100', 0],
 ['0001011011', 0],
 ['1100000110', 0],
 ['1010101011', 0],
 ['0011011110', 0],
 ['1001000101', 0],
 ['1000110000', 0],
 ['0010010000', 0],
 ['0011100110', 0],
 ['1000000110', 0],
 ['1100110000', 0],
 ['1000000110', 0],
 ['1111011110', 0],
 ['0001101111', 0],
 ['1111100100', 0],
 ['1001001001', 0],
 ['1110000000', 0],
 ['1100100110', 0],
 ['1101101111', 0],
 ['0010100110', 0],
 ['1101010101', 0],
 ['0001110100', 0],
 ['0000111001', 0],
 ['1001010100', 0],
 ['1010100111', 0],
 ['1011101010', 0],
 ['1001011010', 0],
 ['0101000000', 0],
 ['1001111110', 0],
 ['0010010011', 0],
 ['0001011010', 0],
 ['1100100110', 0],
 ['1110001001', 0],
 ['1001110101', 0],
 ['1000110111', 0],
 ['1111000110', 0],
 ['1100011000', 0]]

```

▼ Mutate the new population

Mutation is applied to the new population. The mutation rate is typically set quite low (e.g. 0.05). There are several mutation operations that may be applied in this case but change should be quite small.

```

# Function to mutate individuals in the population according to the mutation rate
# Input parameters: A population
# Returns: A modified population
def mutate(pop):
    # Your code goes here
    mutated_pop = []
    for individual in pop:
        original_string = individual[0]
        mutated_list = list(original_string) # Convert string to list of chars

        # Iterate through each bit in the individual's string
        for i in range(len(mutated_list)):

```

